

You Could've Discovered the Yoneda Lemma

An exposition on the Yoneda Lemma that's accessible to those without a math background. The Yoneda Lemma is a very useful statement in Category Theory which is usually communicated as 'you can completely understand something by observing how everything else relate to it'. It has been the subject of intense hype amongst pseudo-intellectual circles surrounding tech and AI, despite how, as we'll see in this post, the statement of the Yoneda Lemma is surprisingly dull.

Author: JuliaPoo
Category: Mathematics
Tags: math, category-theory, yoneda-lemma

Outline

Header 1	2
Header 2	2
Test	2
LK and LJ Calculi	4
LK Calculus Formulation of Classical Logic	4
LJ Calculus Formulation of Intuitionistic Logic	5
Proof Normalisation of LK and LJ Calculus	5
Computational Content	5
Computational Content of LJ Calculus	5
Enforcing CBV evaluation strategy via CPS	7
coLJ Calculus Formulation of co-Intuitionistic Logic	7
Enforcing CBN evaluation strategy via co-CPS	7
Computational Content of coLJ Calculus	7
Wadler's Duality	7
Gentzen's LJ and Closed Cartesian Categories	7
Gentzen's LK and the Continuation Monad	11
CLL and LPC Models	11
Comonad !	11
Monad ?	12
The Chu Construction	12
Bibliography	12

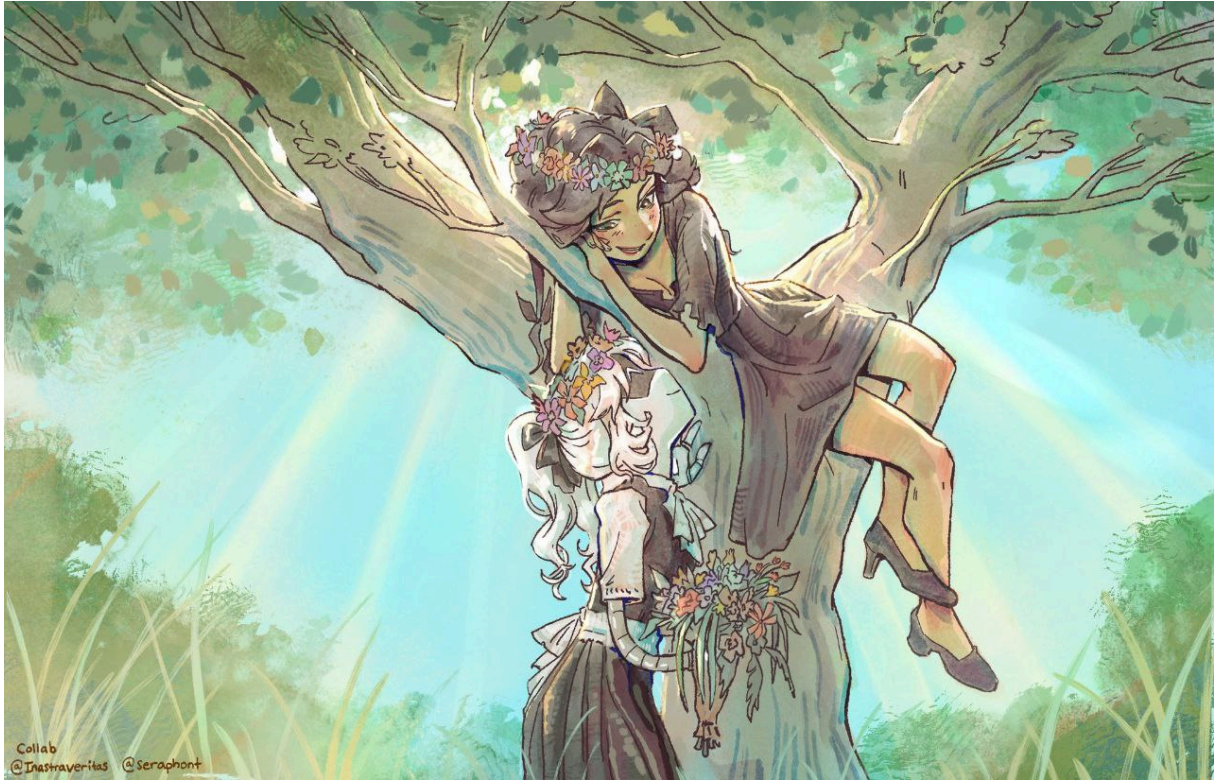


Figure 1: Seraph and Seline's beautiful jessa

Header 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Header 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Test

This is some sample ballalayolehoooo.

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

Also maybe some inline equations hehe like M and $\lim_{n \rightarrow \infty} x_n = 0$ yippie. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Also some more features:

1. List 1
2. List 2
 1. List 2.1
 2. List 2.2
3. List 3

- List 1
- List 2
 - List 2.1
 - List 2.2
- List 3

Code formatting like **bold**, *italics*, raw.

How about codeblocks?

```
def map(xs, f):  
    return [f(x) for x in xs]
```

LK and LJ Calculi

For the LK calculus, we follow the presentation in [1] but make a small adjustment. We define $\neg A := A \rightarrow \perp$ where $\perp$ is the unit of $\wedge$. This allows the $\neg$ -intro rules to be subsumed by $\rightarrow$ -intro rules, and later, to be able to define the LJ calculus as a restriction of LK where each sequent has *exactly* one formula on the right (as opposed to *at least one* in Gentzen's original presentation).

Further, while $\rightarrow$ can be defined as $A \rightarrow B := \neg A \vee B$ in LK, I'll state the $\rightarrow$ rules explicitly as $A \rightarrow B := \neg A \vee B$ does not hold in LJ. This'll allow us to realise LJ as a restriction of LK as mentioned above. To keep LK symmetrical, I'll also state rules for $-$ (co-implication/subtraction).

LK Calculus Formulation of Classical Logic

A sequent in LK calculus is $\Gamma \vdash \Delta$ where both Γ and Δ represent multi-sets of formulas. The derivation rules are as follows:

$$\begin{array}{c}
 \textbf{Core Rules} \\
 \begin{array}{c}
 \Gamma \vdash A, \Delta \quad \Gamma', A \vdash \Delta' \\
 \hline
 \Gamma, \Gamma' \vdash \Delta, \Delta' \quad \text{Cut}
 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \textbf{Structural Rules} \\
 \begin{array}{c}
 \Gamma \vdash \Delta \quad \text{WR} \quad \Gamma \vdash \Delta \quad \text{WL} \quad \Gamma \vdash A, A, \Delta \quad \text{CR} \quad \Gamma, A, A \vdash \Delta \quad \text{CL} \\
 \hline
 \Gamma \vdash \Delta \quad \sigma(\Gamma) \vdash \sigma'(\Delta) \quad \text{Exch}
 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \textbf{Logical Rules} \\
 \begin{array}{c}
 \Gamma \vdash \top, \Delta \quad \top R \quad \text{no } \top L \text{ rule} \quad \text{no } \perp R \text{ rule} \quad \Gamma, \perp \vdash \Delta \quad \perp L \\
 \Gamma \vdash A, \Delta \quad \Gamma \vdash B, \Delta \quad \wedge R \quad \Gamma, A \vdash \Delta \quad \wedge L1 \quad \Gamma, B \vdash \Delta \quad \wedge L2 \\
 \Gamma \vdash A \wedge B, \Delta \quad \Gamma \vdash A, \Delta \quad \Gamma \vdash B, \Delta \quad \vee R1 \quad \Gamma \vdash A \vee B, \Delta \quad \vee R2 \quad \Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta \quad \vee L \\
 \Gamma \vdash A \vee B, \Delta \quad \Gamma \vdash A, \Delta \quad X \text{ free in } \{\Gamma, \Delta\} \quad \forall R \quad \Gamma, A[X := B] \vdash \Delta \quad \forall L \\
 \Gamma \vdash \forall X.A, \Delta \quad \Gamma \vdash A[X := B], \Delta \quad \exists R \quad \Gamma, A \vdash \Delta \quad X \text{ free in } \{\Gamma, \Delta\} \quad \exists L \\
 \Gamma \vdash \exists X.A, \Delta \quad \Gamma \vdash \exists X.A \vdash \Delta
 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \textbf{Derived Rules} \\
 \begin{array}{c}
 \Gamma, A \vdash B, \Delta \quad \rightarrow R \quad \Gamma \vdash A, \Delta \quad \Gamma', B \vdash \Delta' \quad \rightarrow L \\
 \hline
 \Gamma \vdash A \rightarrow B, \Delta \quad \Gamma, \Gamma', A \rightarrow B \vdash \Delta, \Delta' \\
 \Gamma \vdash A, \Delta \quad \Gamma', B \vdash \Delta' \quad -R \quad \Gamma, A \vdash B, \Delta \quad -L \\
 \hline
 \Gamma, \Gamma' \vdash A - B, \Delta, \Delta' \quad \Gamma, A - B \vdash \Delta \\
 \Gamma, A \vdash \Delta \quad \neg R \quad \Gamma \vdash A, \Delta \quad \neg L \\
 \hline
 \Gamma \vdash \neg A, \Delta \quad \Gamma, \neg A \vdash \Delta
 \end{array}
 \end{array}$$

Note. Due to the left and right structural rules, the following two cut rules are equivalent:

$$\begin{array}{c}
 \Gamma \vdash A, \Delta \quad \Gamma', A \vdash \Delta' \\
 \hline
 \Gamma, \Gamma' \vdash \Delta, \Delta' \quad \text{Cut}
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma \vdash A, \Delta \quad \Gamma, A \vdash \Delta \\
 \hline
 \Gamma \vdash \Delta \quad \text{Cut}
 \end{array}$$

The former is used to better connect LK and LJ calculus, as LJ calculus has restricted structural rules. This choice affects the $\rightarrow L$ and $-R$ rules.

LJ Calculus Formulation of Intuitionistic Logic

As mentioned previously, the LJ calculus is a restriction of LK calculus such that all sequences have exactly one formula on the right $\Gamma \vdash A$. This prohibits all right-structural rules and the $\neg R$ rule (hence I omit all $\neg$ rules in LJ). Further, the $\rightarrow$ rule is no longer ‘derived’ and has to be asserted explicitly as the LK theorem $\neg A \vee B \dashv\vdash A \rightarrow B$ no longer holds in LJ. Explicitly, the derivation rules of LJ calculus is as follows:

$$\begin{array}{c}
 \textbf{Core Rules} \\
 \begin{array}{c}
 \Gamma \vdash A \quad \text{Ax} \\
 \Gamma \vdash A \quad \Gamma', A \vdash B \quad \text{Cut} \\
 \Gamma, \Gamma' \vdash B
 \end{array} \\
 \\
 \textbf{Structural Rules} \\
 \begin{array}{c}
 \Gamma \vdash B \quad \text{WL} \\
 \Gamma, A \vdash B \\
 \Gamma, A, A \vdash B \quad \text{CL} \\
 \Gamma, A \vdash B \\
 \Gamma \vdash A \quad \text{Exch} \\
 \sigma(\Gamma) \vdash A
 \end{array} \\
 \\
 \textbf{Logical Rules} \\
 \begin{array}{c}
 \Gamma \vdash \top \quad \top R \quad \text{no } \top L \text{ rule} \quad \text{no } \perp R \text{ rule} \\
 \Gamma, \perp \vdash A \quad \perp L \\
 \Gamma \vdash A \quad \Gamma \vdash B \quad \wedge R \\
 \Gamma \vdash A \wedge B \\
 \Gamma, A \vdash C \quad \wedge L1 \\
 \Gamma, A \wedge B \vdash C \\
 \Gamma, B \vdash C \quad \wedge L2 \\
 \Gamma, A \wedge B \vdash C \\
 \Gamma \vdash A \quad \vee R1 \\
 \Gamma \vdash A \vee B \\
 \Gamma \vdash B \quad \vee R2 \\
 \Gamma, A \vee B \vdash C \\
 \Gamma, A \vdash C \quad \Gamma, B \vdash C \quad \vee L \\
 \Gamma, A \vee B \vdash C \\
 \Gamma \vdash A \quad X \text{ free in } \Gamma \quad \forall R \\
 \Gamma, A[X := B] \vdash C \quad \forall L \\
 \Gamma, \forall X.A \vdash C \\
 \Gamma \vdash A[X := B] \quad \exists R \\
 \Gamma, A \vdash B \quad X \text{ free in } \Gamma \quad \exists L \\
 \Gamma, \exists X.A \vdash B \\
 \Gamma, A \vdash B \quad \rightarrow R \\
 \Gamma \vdash A \rightarrow B \\
 \Gamma \vdash A \quad \Gamma', B \vdash C \quad \rightarrow L \\
 \Gamma, \Gamma', A \rightarrow B \vdash C
 \end{array} \\
 \\
 \textbf{Derived Rules} \\
 \begin{array}{c}
 \Gamma, A \vdash \perp \quad \neg R \\
 \Gamma \vdash \neg A \\
 \Gamma \vdash A \quad \neg L \\
 \Gamma, \neg A \vdash \perp
 \end{array}
 \end{array}$$

Proof Normalisation of LK and LJ Calculus

β and η reduction, and cut-free proofs

Computational Content

The computational interpretation of a sequent $\Gamma \vdash \Delta$ is a set of *producers* Γ and a set of *consumers* Δ . Since Γ is conjunctive and Δ is disjunctive, a program of type $\Gamma \vdash \Delta$ can be interpreted as a set of resources Γ that fulfills some subset of obligations Δ .

Computational Content of LJ Calculus

Note: This entire section is just my intuition, it could be wrong in subtle ways

Since LJ calculus contain exactly one formula on the right, the unique *consumer* can be thought of as the ‘goal’ of the computation (or return value). Further, the producers can be interpreted as typed holes.

Since the consumer/goal is unique, we can build the proof-term (program) by annotating the goal. The ‘initialising’ of the holes can be done via a let statement. I’ll expound on this interpretation to establish a better connection between LJ and lambda calculus. Later, System L for LK calculus will be introduced as an extension of this idea.

Concretely, the computational interpretation is as follows:

Syntax

$$\begin{aligned} A, B \in \text{Types} &:= \top \mid \perp \mid A \times B \mid A + B \mid A \rightarrow B \mid \forall X. A \mid \exists X. A \\ N, M, P \in \text{Terms} &:= x \mid () \mid \text{let } x = N \text{ in } M \mid (N, M) \mid \pi_1 N \mid \pi_2 N \mid \text{inr } N \mid \text{inl } N \\ &\mid \text{case } N \text{ of } \{\text{inr } x \Rightarrow M \mid \text{inl } y \Rightarrow P\} \mid \lambda x. N \mid NM \mid \Lambda A. N \\ &\mid NA \mid A @ M \mid \text{case } N \text{ of } \{A @ x \Rightarrow N\} \end{aligned}$$

Core Rules

$$\begin{array}{c} \Gamma \vdash t : A \quad \Gamma', x : A \vdash t' : B \\ x : A \vdash x : A \quad \Gamma, \Gamma' \vdash \text{let } x = t \text{ in } t' : B \end{array} \begin{array}{c} \text{Ax} \\ \text{Cut} \end{array}$$

Structural Rules

$$\begin{array}{c} \Gamma \vdash t : B \\ \Gamma, x : A \vdash t : B \end{array} \text{WL} \quad \begin{array}{c} \Gamma, x : A, x' : A \vdash t : B \\ \Gamma, x : A \vdash t[x' := x] : B \end{array} \text{CL} \quad \begin{array}{c} \Gamma \vdash t : A \\ \sigma(\Gamma) \vdash t : A \end{array} \text{Exch}$$

Logical Rules

$$\begin{array}{c} \Gamma \vdash () : \top \quad \top R \quad \text{no } \top L \text{ rule} \quad \text{no } \perp R \text{ rule} \quad \Gamma, x : \perp \vdash \text{absurd}(x) : A \quad \perp L \\ \Gamma \vdash t : A \quad \Gamma \vdash t' : B \\ \Gamma \vdash (t, t') : A \wedge B \quad \wedge R \\ \Gamma, x : A \vdash t : C \quad \Gamma, y : B \vdash t : C \\ \Gamma, p : A \wedge B \vdash \text{let } x = \pi_1 p \text{ in } t : C \quad \wedge L1 \quad \Gamma, p : A \wedge B \vdash \text{let } y = \pi_2 p \text{ in } t : C \quad \wedge L2 \\ \Gamma \vdash t : A \quad \Gamma \vdash t : B \\ \Gamma \vdash \text{inl } t : A \vee B \quad \vee R1 \quad \Gamma \vdash \text{inr } t : A \vee B \quad \vee R2 \\ \Gamma, x : A \vdash t : C \quad \Gamma, y : B \vdash t' : C \\ \Gamma, s : A \vee B \vdash \text{case } s \text{ of } \{\text{inr } x \Rightarrow t \mid \text{inl } y \Rightarrow t'\} : C \quad \vee L \\ \Gamma, x : A \vdash t : B \quad \Gamma \vdash t : A \quad \Gamma', x : B \vdash t' : C \\ \Gamma \vdash \lambda x. t : A \rightarrow B \quad \rightarrow R \quad \Gamma, \Gamma', f : A \rightarrow B \vdash \text{let } x = ft \text{ in } t' : C \quad \rightarrow L \\ \Gamma \vdash t : A \quad X \text{ free in } \Gamma \quad \Gamma, x : A[X := B] \vdash t : C \\ \Gamma \vdash \Lambda X. t : \forall X. A \quad \forall R \quad \Gamma, e : \forall X. A \vdash \text{let } x = eB \text{ in } t : C \quad \forall L \\ \Gamma \vdash t : A[X := B] \quad \Gamma, x : A \vdash t : B \quad X \text{ free in } \Gamma \\ \Gamma \vdash B @ t : \exists X. A \quad \exists R \quad \Gamma, e : \exists X. A \vdash \text{case } e \text{ of } \{B @ x \Rightarrow t\} : B \quad \exists L \end{array}$$

Derived Rules

$$\begin{array}{c} \Gamma, x : A \vdash t : \perp \\ \Gamma \vdash \lambda x. t : \neg A \end{array} \neg R \quad \begin{array}{c} \Gamma \vdash t : A \\ \Gamma, f : \neg A \vdash ft : \perp \end{array} \neg L$$

β -Reduction

$$\begin{array}{ll} \beta_{\times 1} : \text{let } p = (t, t') \text{ in let } x = \pi_1 p \text{ in } t'' & \rightsquigarrow \text{let } x = t \text{ in } t'' \\ \beta_{\times 2} : \text{let } p = (t, t') \text{ in let } y = \pi_2 p \text{ in } t'' & \rightsquigarrow \text{let } y = t' \text{ in } t'' \\ \beta_{+1} : \text{let } s = \text{inr } t \text{ in case } s \text{ of } \{\text{inr } x \Rightarrow t' \mid \text{inl } y \Rightarrow t''\} & \rightsquigarrow \text{let } x = t \text{ in } t' \\ \beta_{+2} : \text{let } s = \text{inl } t \text{ in case } s \text{ of } \{\text{inr } x \Rightarrow t' \mid \text{inl } y \Rightarrow t''\} & \rightsquigarrow \text{let } y = t \text{ in } t'' \\ \beta_{\rightarrow} : \text{let } f = \lambda x. t \text{ in let } y = ft' \text{ in } t'' & \rightsquigarrow \text{let } y = t[x := t'] \text{ in } t'' \\ \beta_{\forall} : \text{let } e = \Lambda X. t \text{ in let } x = eB \text{ in } t' & \rightsquigarrow \text{let } x = t[X := B] \text{ in } t' \\ \beta_{\exists} : \text{let } e = B @ t \text{ in case } e \text{ of } \{B @ x \Rightarrow t'\} & \rightsquigarrow \text{let } x = t \text{ in } t' \end{array}$$

Left-intro rules instantiate existing holes by destructing fresh holes while the right-intro rules construct terms. This is exactly the usual intro-elim rules in lambda calculus (or Gentzen's NJ calculus). Hence, the LJ calculus can be seen as a type system for (polymorphic) lambda calculus.

In particular, each elimination rule can be derived by pairing a left-intro rule with a cut and eliminating the superfluous let statements:

$$\frac{\overbrace{\Gamma \vdash p : A \wedge B}^{\pi} \quad \overbrace{p : A \wedge B \vdash \text{let } x = \pi_1 p \text{ in } x : C}^{Ax} \quad \wedge L1}{\Gamma \vdash \text{let } p = p \text{ in let } x = \pi_1 p \text{ in } x : C} \text{Cut} \implies \frac{\overbrace{\Gamma \vdash p : A \times B}^{\pi} \quad \wedge\text{-elim1}}{\Gamma \vdash \pi_1 p : A}$$

More concretely, the NJ calculus can be described as follows:

TODO

NJ and LJ have equivalent proving power. I.e., a proof of NJ can be translated into LJ and vice-versa.

The difference between LJ calculus and NJ calculus is that LJ is explicit about the typed holes while NJ keeps the holes implicit. More concretely, using the substitution-based Krivine-style machine, NJ calculus has the following CBN operational semantics, which can be translated into an LJ proof-term, making the evaluation context explicit (only $\rightarrow$ rules are shown):

$$\begin{aligned} \langle vv' \parallel E \rangle \rightsquigarrow \langle v \parallel E[\Box v'] \rangle &\implies \llbracket vv' \rrbracket \rightsquigarrow \text{let } f = \llbracket v \rrbracket \text{ in } f v' \\ \langle \lambda x.v \parallel E[\Box v'] \rangle \rightsquigarrow \langle v[x := v'] \parallel E \rangle &\implies \text{let } f = \llbracket \lambda x.v \rrbracket \text{ in } f v' \rightsquigarrow \llbracket v[x := v'] \rrbracket \end{aligned}$$

The first rule splits the term vv' into a term and a context. The second rule is equivalent to $\beta_{\rightarrow}$ written above.

In particular, we could transform an NJ proof term by applying the translation above. The process is KINDA like a translation of NJ into LJ and *then* evaluating the LJ term via CBN semantics. Just that the two steps are interweaved. I don't know what I'm talking about.

This seems similar to the vibe of [2] where perhaps LJ can be seen as a more explicit 'intermediate language'.

Enforcing CBV evaluation strategy via CPS

coLJ Calculus Formulation of co-Intuitionistic Logic

Enforcing CBN evaluation strategy via co-CPS

Computational Content of coLJ Calculus

Wadler's Duality

Gentzen's LJ and Closed Cartesian Categories

We'll focus on the propositional fragment of the LJ calculus.

TODO: Expound on LJ without negation, BCC semantics, and then the problem of adding negation to motivate Pointed LJ

For non-trivial computational semantics, we'll be using a slightly modified definition of negation from the LJ calculus. In particular, we'll define negation $\neg_r A := A \rightarrow r$ where r is a distinguished type that's generally not equal to $\perp$. The modified calculus, which we denote as pointed-LJ calculus, is concretely as follows:

Definition 1 (Pointed-LJ).

Formulas

$$A, B ::= p \mid r \mid \perp \mid \top \mid A \vee B \mid A \wedge B \mid A \rightarrow B$$

Core Rules

$$\begin{array}{c} A \vdash A \quad \text{Ax} \\ \Gamma \vdash A \quad \Gamma', A \vdash B \\ \hline \Gamma, \Gamma' \vdash B \quad \text{Cut} \end{array}$$

Structural Rules

$$\begin{array}{c} \Gamma \vdash B \\ \hline \Gamma, A \vdash B \quad \text{WL} \end{array} \quad \begin{array}{c} \Gamma, A, A \vdash B \\ \hline \Gamma, A \vdash B \quad \text{CL} \end{array} \quad \begin{array}{c} \Gamma \vdash A \\ \hline \sigma(\Gamma) \vdash A \quad \text{Exch} \end{array}$$

Logical Rules

$$\begin{array}{c} \Gamma \vdash \top \quad \top R \quad \text{no } \top L \text{ rule} \quad \text{no } \perp R \text{ rule} \quad \Gamma, \perp \vdash A \quad \perp L \\ \Gamma \vdash A \quad \Gamma \vdash B \quad \wedge R \quad \Gamma, A \vdash C \quad \wedge L1 \quad \Gamma, B \vdash C \quad \wedge L2 \\ \hline \Gamma \vdash A \wedge B \quad \Gamma, A \wedge B \vdash C \\ \Gamma \vdash A \quad \vee R1 \quad \Gamma \vdash B \quad \vee R2 \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C \quad \vee L \\ \hline \Gamma \vdash A \vee B \quad \Gamma, A \vee B \vdash C \\ \Gamma, A \vdash B \quad \rightarrow R \quad \Gamma \vdash A \quad \Gamma', B \vdash C \quad \rightarrow L \\ \hline \Gamma \vdash A \rightarrow B \quad \Gamma, \Gamma', A \rightarrow B \vdash C \end{array}$$

Derived Rules

$$\begin{array}{c} \Gamma, A \vdash r \\ \hline \Gamma \vdash \neg_r A \quad \neg_r R \end{array} \quad \begin{array}{c} \Gamma \vdash A \\ \hline \Gamma, \neg_r A \vdash r \quad \neg_r L \end{array}$$

Where p is a ground-type and r is the distinguished object representing a result type. Pointed-LJ can be seen as an extension of propositional LJ with a distinguished type r (hence the name ‘pointed-LJ’, a reference to pointed-sets). LJ can be recovered from a pointed-LJ by setting $r = \perp$. Hence, as will be later expounded, the usual soundness proof for LJ wrt a BCC is directly applicable for the soundness proof for LJ wrt a *pointed* BCC (A BCC $\mathcal{C}$ with a distinguished object $R \in \text{obj}(\mathcal{C})$).

Since an LJ sequent can be embedded directly into pointed-LJ ($\neg_{\perp} A$ embeds as $A \rightarrow \perp$ and not $A \rightarrow r := \neg_r A$), we’ve not lost any proving power by moving to pointed-LJ. However, since taking negation to be $\neg_r$ instead of $\neg_{\perp}$ is intended to preserve non-degenerate computational semantics, a natural question is to ask if every LJ sequent admits a non-degenerate computational semantics as a pointed-LJ sequent. This question is intentionally left vague as the author does not have a good idea of what she’s writing about, but there’s a decently precise way to describe the negative result.

Lemma 2. Define $(-)^{\dagger}$ transforming an LJ sequent into a pointed-LJ sequent such that it replaces all instances of $\perp$ with r . Then, $(-)^{\dagger}$ preserves derivability iff for all LJ formulas A , $r \vdash A^{\dagger}$ is derivable.

Proof. Assume $\forall A, r \vdash A^{\dagger}$ is derivable. Given a derivable LJ sequent s with proof π , we can show that $s^{\dagger}$ is derivable if for all sequents $t \in \pi$, $t^{\dagger}$ is provable in pointed-LJ. The only rule that’s changed by $(-)^{\dagger}$ is the $\perp L$ rule. I.e.,

$$\left(\Gamma, \perp \vdash A \quad \perp L \right)^{\dagger} = \Gamma^{\dagger}, r \vdash A^{\dagger} \quad \perp L$$

The RHS is derivable by assumption that $\forall A, r \vdash A^{\dagger}$ is derivable.

Conversely, let’s assume $\forall s$ provable in LJ, $s^{\dagger}$ provable in pointed-LJ. Consider the provable LJ sequent $s := [\perp \vdash A]$ via explosion ($\perp L$ rule). Then $s^{\dagger} = [r \vdash A^{\dagger}]$ is derivable. ■

Note. This is insufficient to conclude that $r \dashv\vdash \perp$ since any cut-free proof of $r \vdash \perp$ satisfies the subformula property, and going through every possible structural and introduction rules, none of the rules can produce the sequent $r \vdash \perp$.

By the above lemma, the condition $\forall A, r \vdash A^\dagger$ is too strict for an atomic r for non-trivial computational semantics. In particular, we encounter the same problem as before with regards to CPS translation, since r behaves like $\perp$ for formulas in the image of $(-)^{\dagger}$.

TODO: Expound on the exact problem lmao

Hence, we reject the assumption $\forall A, r \vdash A^\dagger$. As a consequence, for an LJ proof π , $\pi^\dagger$ is a valid proof in pointed-LJ iff π does not use $\perp L$ rule. One way to think of this is that, if our LJ program contains the absurd construct, it has degenerate semantics in pointed-LJ, which is totally expected.

In order to highlight this distinction, we'll change the notation of pointed-LJ slightly by denoting r as $\perp_r$, $\perp$ as 0 and $\top$ as 1 to highlight that $\perp$ no longer plays the role of false but rather, the unit of $\wedge$:

Definition 3 (Pointed-LJ).

$$\begin{array}{c}
\textbf{Formulas} \\
A, B ::= p \mid \perp_r \mid 0 \mid 1 \mid A \vee B \mid A \wedge B \mid A \rightarrow B \\
\\
\textbf{Core Rules} \\
\begin{array}{c}
\Gamma \vdash A \quad \Gamma', A \vdash B \\
\hline
\Gamma, \Gamma' \vdash B \quad \text{Cut}
\end{array}
\quad
\begin{array}{c}
A \vdash A \quad \text{Ax}
\end{array}
\\
\\
\textbf{Structural Rules} \\
\begin{array}{c}
\Gamma \vdash B \\
\hline
\Gamma, A \vdash B \quad \text{WL}
\end{array}
\quad
\begin{array}{c}
\Gamma, A, A \vdash B \\
\hline
\Gamma, A \vdash B \quad \text{CL}
\end{array}
\quad
\begin{array}{c}
\Gamma \vdash A \\
\hline
\sigma(\Gamma) \vdash A \quad \text{Exch}
\end{array}
\\
\\
\textbf{Logical Rules} \\
\begin{array}{c}
\Gamma \vdash 1 \quad \text{1R} \quad \text{no 1L rule} \quad \text{no 0R rule} \quad \Gamma, 0 \vdash A \quad \text{0L} \\
\Gamma \vdash A \quad \Gamma \vdash B \quad \wedge R \quad \Gamma, A \vdash C \quad \wedge L1 \quad \Gamma, B \vdash C \quad \wedge L2 \\
\Gamma \vdash A \wedge B \quad \Gamma \vdash A \wedge B \vdash C \\
\Gamma \vdash A \quad \Gamma \vdash B \quad \vee R1 \quad \Gamma \vdash B \quad \vee R2 \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C \quad \vee L \\
\Gamma \vdash A \vee B \quad \Gamma \vdash A \vee B \vdash C \\
\Gamma, A \vdash B \quad \rightarrow R \quad \Gamma \vdash A \quad \Gamma', B \vdash C \quad \rightarrow L \\
\Gamma \vdash A \rightarrow B \quad \Gamma, \Gamma', A \rightarrow B \vdash C
\end{array}
\\
\\
\textbf{Derived Rules} \\
\begin{array}{c}
\Gamma, A \vdash \perp_r \quad \neg_r R \quad \Gamma \vdash A \quad \neg_r L \\
\Gamma \vdash \neg_r A \quad \Gamma, \neg_r A \vdash \perp_r
\end{array}
\end{array}$$

TODO: Bridge gap to denotational semantics

Definition 4 (Bicartesian Closed Category). A bicartesian closed category (BCC) is a category $\mathcal{C}$ possessing the following adjoints:

$$\sum_{i=1}^n (-_i) \dashv \Delta_n \dashv \prod_{i=1}^n (-_i) \quad \forall n \in \mathbb{Z}_{\geq 0} \quad (-) \times A \dashv (-)^A \quad \forall A \in \text{obj}(\mathcal{C})$$

Where $\Delta_n : \mathcal{C} \rightarrow \mathcal{C}^n$ is the diagonal functor $A \mapsto (A, \dots, A)_{\text{Cat}}$, where $(\dots)_{\text{Cat}}$ is the pairing in Cat and Π_i the corresponding projections.

The usual canonical maps typical in presentations of BCCs can be constructed from the above datum as follows:

$$\begin{aligned}
(f, g) : A \rightarrow X \times Y &:= (f \times g) \circ \eta_{\times, A} \quad \text{for } f : A \rightarrow X, g : A \rightarrow Y \\
\pi_1 : X \times Y \rightarrow X &:= \Pi_1 \circ \varepsilon_{\times, X \times Y} \\
\pi_2 : X \times Y \rightarrow Y &:= \Pi_2 \circ \varepsilon_{\times, X \times Y} \\
[f, g] : X + Y \rightarrow A &:= \varepsilon_{+, A} \circ (f + g) \quad \text{for } f : X \rightarrow A, g : Y \rightarrow A \\
\text{inl} : X \rightarrow X + Y &:= \Pi_1 \circ \eta_{+, X + Y} \\
\text{inr} : Y \rightarrow X + Y &:= \Pi_2 \circ \eta_{+, X + Y} \\
\lambda(f) : X \rightarrow Y^A &:= f^A \circ \eta_{\rightarrow, A, X} \quad \text{for } f : X \times A \rightarrow Y \\
\text{ev}_{A, X} : X^A \times A \rightarrow X &:= \varepsilon_{\rightarrow, A, X}
\end{aligned}$$

Where

- $\eta_{\times}$ and $\varepsilon_{\times}$ are the unit and counit of the adjunction $\Delta_2 \dashv (-) \times (-)$.
- η_{+} and ε_{+} are the unit and counit of the adjunction $(-) + (-) \dashv \Delta_2$.
- $\eta_{\rightarrow, A}$ and $\varepsilon_{\rightarrow, A}$ are the unit and counit of the adjunction $(-) \times A \dashv (-)^A$.

To define the units 0 and 1, denote functors F, G such that $F \dashv \Delta_0 \dashv G$. Since $\text{dom}(G) = \text{dom}(F)$ is the terminal category $\mathbb{1}$ with unique object $\star \in \text{obj } \mathbb{1}$, we can define $1 := G(\star)$ and $0 := F(\star)$.

A pointed-LJ formula A embeds into into a pointed BCC $(\mathcal{C}, R)$ as follows:

- Ground type A assigned an object $\llbracket A \rrbracket \in \mathcal{C}$.
- $\llbracket \perp \rrbracket = 0, \llbracket \top \rrbracket = 1, \llbracket r \rrbracket = R$
- $\llbracket A \wedge B \rrbracket := \llbracket A \rrbracket \times \llbracket B \rrbracket$
- $\llbracket A \vee B \rrbracket := \llbracket A \rrbracket \vee \llbracket B \rrbracket$
- $\llbracket A \rightarrow B \rrbracket := \llbracket B \rrbracket^{\llbracket A \rrbracket}$

Where $R \in \text{obj}(\mathcal{C})$ is a distinguished object representing the ‘result type’. For non-trivial computational semantics of the embedding, R is taken to be non-trivial ($R \neq 0$). If $R = 0$, as used in the embedding $\llbracket \neg A \rrbracket := \llbracket A \rightarrow \perp \rrbracket$, the continuation monad (discussed later) collapses into a monad mapping all morphisms $A \rightarrow B$ into $! : A \rightarrow 0^{0^B} \cong 1$.

A pointed-LJ sequent $\Gamma \vdash A$ embeds into $(\mathcal{C}, R)$ as a morphism $\llbracket \Gamma \vdash A \rrbracket : \prod_{B \in \Gamma} \llbracket B \rrbracket \rightarrow \llbracket A \rrbracket$ as follows:

$$\begin{aligned}
\llbracket A \vdash A \rrbracket &:= \text{id}_A = \lambda x. x \\
\llbracket \Gamma, \Gamma' \vdash B \rrbracket &:= \llbracket \Gamma \rrbracket \times \llbracket \Gamma' \rrbracket \xrightarrow{\llbracket \Gamma \vdash A \rrbracket \times \text{id}} \llbracket A \rrbracket \times \llbracket \Gamma' \rrbracket \xrightarrow{\llbracket \Gamma', A \vdash B \rrbracket} \llbracket B \rrbracket = \lambda \gamma' \gamma. h'(h(\gamma), \gamma') \sim \lambda \gamma. h'(h(\gamma)) \\
\llbracket \Gamma, A \vdash B \rrbracket &:= \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \xrightarrow{\pi} \llbracket \Gamma \rrbracket \xrightarrow{\llbracket \Gamma \vdash B \rrbracket} \llbracket B \rrbracket = \lambda \gamma a. h\gamma \sim \lambda a. h \\
\llbracket \Gamma, A \vdash B \rrbracket &:= \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \xrightarrow{\Delta} \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \times \llbracket A \rrbracket \xrightarrow{\llbracket \Gamma, A, A \vdash B \rrbracket} \llbracket B \rrbracket = \lambda \gamma a. h(a, a, \gamma) \sim \lambda a. haa \\
\llbracket \sigma(\Gamma) \vdash A \rrbracket &:= \llbracket \sigma(\Gamma) \rrbracket \xrightarrow{\sigma} \llbracket \Gamma \rrbracket \xrightarrow{\llbracket \Gamma \vdash A \rrbracket} \llbracket A \rrbracket = \lambda \sigma(\gamma). h(\gamma) \\
\llbracket \Gamma \vdash \top \rrbracket &:= \llbracket \Gamma \rrbracket \xrightarrow{!} 1 = \lambda \gamma. () \\
\llbracket \Gamma, \perp \vdash A \rrbracket &:= \llbracket \Gamma \rrbracket \times 0 \xrightarrow{\pi} 0 \xrightarrow{!} A = \lambda \gamma b. \text{absurd}(b) \\
\llbracket \Gamma \vdash A \wedge B \rrbracket &:= \llbracket \Gamma \rrbracket \xrightarrow{\langle \llbracket \Gamma \vdash A \rrbracket, \llbracket \Gamma \vdash B \rrbracket \rangle} \llbracket A \rrbracket \times \llbracket B \rrbracket = \lambda \gamma. \langle h\gamma, h'\gamma' \rangle \\
\llbracket \Gamma, A \wedge B \vdash C \rrbracket &:= \llbracket \Gamma \rrbracket \times (\llbracket A \rrbracket \times \llbracket B \rrbracket) \xrightarrow{\pi} \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \xrightarrow{\llbracket \Gamma, A \vdash C \rrbracket} \llbracket C \rrbracket = \lambda \gamma ab. h(a, \gamma) \sim \lambda ab. ha \\
\llbracket \Gamma, A \wedge B \vdash C \rrbracket &:= \llbracket \Gamma \rrbracket \times (\llbracket A \rrbracket \times \llbracket B \rrbracket) \xrightarrow{\pi} \llbracket \Gamma \rrbracket \times \llbracket B \rrbracket \xrightarrow{\llbracket \Gamma, B \vdash C \rrbracket} \llbracket C \rrbracket = \lambda \gamma ab. h(b, \gamma) \sim \lambda ab. hb \\
\llbracket \Gamma \vdash A \vee B \rrbracket &:= \llbracket \Gamma \rrbracket \xrightarrow{\llbracket \Gamma \vdash A \rrbracket} \llbracket A \rrbracket \xrightarrow{\text{inl}} \llbracket A \rrbracket + \llbracket B \rrbracket = \lambda \gamma. \text{inl}(h(\gamma)) \\
\llbracket \Gamma \vdash A \vee B \rrbracket &:= \llbracket \Gamma \rrbracket \xrightarrow{\llbracket \Gamma \vdash B \rrbracket} \llbracket B \rrbracket \xrightarrow{\text{inr}} \llbracket A \rrbracket + \llbracket B \rrbracket = \lambda \gamma. \text{inr}(h(\gamma)) \\
\llbracket \Gamma, A \vee B \vdash C \rrbracket &:= \llbracket \Gamma \rrbracket \times (\llbracket A \rrbracket + \llbracket B \rrbracket) \cong (\llbracket \Gamma \rrbracket \times \llbracket A \rrbracket) + (\llbracket \Gamma \rrbracket \times \llbracket B \rrbracket) \xrightarrow{\llbracket \llbracket \Gamma, A \vdash C \rrbracket, \llbracket \Gamma, B \vdash C \rrbracket \rrbracket} \llbracket C \rrbracket \\
&= \lambda \gamma s. \text{case } s \text{ of } \{ \text{inl}(a) \Rightarrow ha\gamma \mid \text{inr}(b) \Rightarrow h'b\gamma \} \\
\llbracket \Gamma \vdash A \rightarrow B \rrbracket &:= \llbracket \Gamma \rrbracket \xrightarrow{\lambda(\llbracket \Gamma, A \vdash B \rrbracket)} \llbracket B \rrbracket^{\llbracket A \rrbracket} = \lambda \gamma. \lambda a. h(a, \gamma) \sim h \\
\llbracket \Gamma, \Gamma', A \rightarrow B \vdash C \rrbracket &:= \llbracket \Gamma \rrbracket \times \llbracket \Gamma' \rrbracket \times \llbracket B \rrbracket^{\llbracket A \rrbracket} \xrightarrow{\llbracket \Gamma \vdash A \rrbracket \times \text{id}} \llbracket \Gamma' \rrbracket \times \llbracket A \rrbracket \times \llbracket B \rrbracket^{\llbracket A \rrbracket} \xrightarrow{\text{ev}} \llbracket \Gamma' \rrbracket \times \llbracket B' \rrbracket \xrightarrow{\llbracket \Gamma', B' \vdash C \rrbracket} \llbracket C \rrbracket \\
&= \lambda \gamma' \gamma f. h'(f(h\gamma), \gamma') \sim \lambda \gamma f. h'(f(h\gamma))
\end{aligned}$$

Where

- $\Delta : A \rightarrow A \times A$ is the diagonal endofunctor,
- σ is the braiding isomorphism for $\times$,
- $!$ denotes the unique morphism $A \rightarrow 1$ or $0 \rightarrow A$,

Note. Via cut elimination over proofs in LJ, the cut formula is actually not needed in the definition of the embedding. The above embedding being well-defined over proof reduction is the concern of the soundness theorem that'll be discussed below.

TODO: Talk about internal language, canonicity, (LJ, NJ), and present the lambda calculi thingy

TODO: We can't talk about embedding a sequent. This is ill-defined for a pointed-LJ. We can only talk about embedding a *proof* and it being well-defined over β/η reduction.

Gentzen's LK and the Continuation Monad

TODO: Pointed LJ avoids the collapse of CBV semantics of $\wedge$

CLL and LPC Models

Comonad !

KabazAAM

Monad ?

The Chu Construction

Bibliography

- [1] P. DOWNEN and Z. M. ARIOLA, “A tutorial on computational classical logic and the sequent calculus,” *Journal of Functional Programming*, vol. 28, p. e3, 2018, doi: 10.1017/S0956796818000023.
- [2] P. Downen, L. Maurer, Z. M. Ariola, and S. Peyton Jones, “Sequent calculus as a compiler intermediate language,” *SIGPLAN Not.*, vol. 51, no. 9, pp. 74–88, Sept. 2016, doi: 10.1145/3022670.2951931.